ZitPit: Consumer-Side Admission Control for Agentic Software Intake

Turning First-Seen External Artifacts into Policy Events

Jepson Taylor Chris Brousseau Jordan Hildebrandt Kelli Quinn
jepson@veox.ai chris@veox.ai j@veox.ai kelli@veox.ai
VEOX Research Group
https://github.com/jeppsontaylor/zitpit

Abstract

AI IDEs and coding agents compress discovery, fetch, workspace open, installation, and execution into one low-observability loop. Existing defenses such as provenance frameworks, package and repository firewalls, runtime protection, and tool-approval prompts each cover part of that path, but they often leave the final consumer-side execution decision implicit. ZitPit is a 100% open-source Rust system that argues for a stricter boundary: first-seen external artifacts should become durable policy events before they gain execution rights on protected developer or CI hosts. The current public evidence is intentionally narrow and explicit. It includes repeated Git smart-HTTP intake measurements showing that approved artifacts can remain faster than unmanaged public fetch, plus implemented protected-session and governed-egress proof families. The broader contribution is architectural rather than universal-coverage-by-assertion: ZitPit unifies artifact admission, repo-open state, capability-scoped execution, and durable policy records at the consumer execution boundary for agentic workflows.

Index Terms—software supply chain security, AI agents, admission control, provenance, quarantine, governed execution, observability

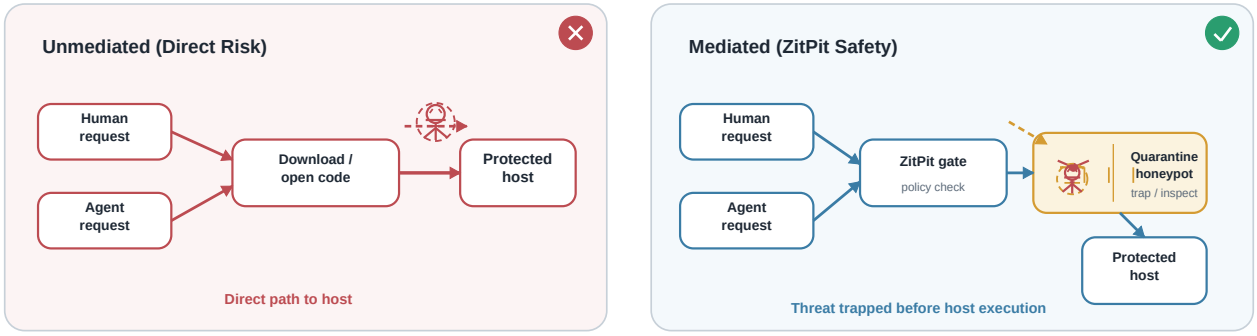

Fig. 1. Without mediation, human and agent requests can move newly encountered external software directly toward host execution; with ZitPit, first-seen artifacts are held behind policy check, capability grant, and quarantine when required.

I. Introduction

Agentic development has changed the tempo of software intake. A developer can ask an AI IDE to "set up this repo," and the tool may immediately clone a repository, open project memory files, attach MCP servers, evaluate workspace configuration, install dependencies, run build hooks, and invoke helper scripts before a human reaches a durable review checkpoint. The security problem is no longer only that a package might be malicious. It is that discovery, fetch, repo-open state, installation, and execution now collapse into one conversational loop with weak artifact-level observability.

This is visible in current tooling. Claude Code documentation covers npm installation, MCP configuration, hooks, project memory, devcontainers, and settings that shape tool use or execution surfaces [1]–[6]. GitHub Actions guidance insists that third-party actions should be pinned to immutable SHAs because mutable references shift trust exactly where automation is strongest [7], [8]. VS Code Workspace Trust likewise exists because opening a folder can enable tasks, settings, extensions, and agents that influence execution [9], [10].

The resulting gap is consumer-side. Provenance systems can describe identity and build lineage. Repository and package firewalls can screen particular ecosystems. Runtime protection can watch after code begins executing. Command approval prompts can limit some tool uses. But in agentic workflows these controls still often stop short of one durable question: *has this exact first-seen external artifact earned execution rights on this protected host, under this policy, in this context?* [11]–[18]

> **Thesis.** ZitPit is a consumer-side software admission control layer for agentic development. First-seen external artifacts must earn execution rights under durable policy before they affect a protected host.

The paper's contribution is not the claim that ZitPit already closes every package manager, IDE, workflow engine, or runtime path. The contribution is narrower and more defensible: ZitPit identifies a missing enforcement boundary for agentic development and shows preliminary public evidence that this boundary can be made visible, policy-scoped, and fast enough to be deployable.

II. Why Admission Control Matters

The importance of this boundary is architectural. Provenance only becomes operational when a consumer-side system uses it to decide whether execution rights will be granted. Repo-open state matters because opening a repository can change

tool behavior before durable review. Capability-scoped verdicts matter because fetch, build, test, and host execution are different trust decisions. The safe path must be faster than unmanaged public fetch because mandatory controls that lose on latency are routinely bypassed.

Novelty claim. ZitPit does not claim to invent application control, sandboxing, or provenance. Its novelty is the combination of artifact admission, repo-open state, capability-scoped execution, and durable policy events at the consumer execution boundary for agentic workflows.

This matters especially for smaller organizations. Large enterprises can sometimes absorb fragmented controls through internal mirrors, release engineering, and custom policy teams. Smaller teams face the same machine-speed intake dynamics without the same guardrails. OpenSSF's guidance for AI code assistants emphasizes prompt injection, package confusion, and automation risk for exactly this reason [19].

III. Current Proof Boundary

The strongest way to present ZitPit is to separate current proof from future ambition. Table I summarizes the current public evidence boundary that this paper relies on.

What ZitPit does not claim. ZitPit is not a general agent-safety system. It does not prove that unknown software is benign, does not claim full ecosystem closure today, does not make trusted-publisher compromise harmless, and does not claim safety for unsupported or unmanaged paths.

IV. Threat Model and Definitions

Definitions. An *artifact* is the external code object or repo-scoped execution bundle being mediated: a Git ref target, package tarball, wheel, crate source, workflow action, raw installer payload, or repo-open configuration surface. *First-seen* means first observed by the ZitPit-mediated trust domain for a given immutable identity. A *policy event* is the durable record created when a selector is resolved, evaluated, and granted or denied capability. A *protected host* is a developer or CI environment whose execution rights are gated by ZitPit.

A. Mandatory Mediation and Bypass Assumptions

The current guarantee depends on mandatory mediation or transparent redirection for the paths under protection. This is the hardest part of the architecture and the place where skeptical reviewers will press first. Git submodules, partial clone follow-on fetches, Git LFS hydration, browser downloads, vendored tarballs, alternate registries, local copies, and direct unmanaged egress all weaken the guarantee if they escape policy visibility [22]–[26]

B. Transitive Closure and Delayed Resolution

Top-level pinning is not enough when transitive dependencies, build backends, workflow reuse, or devcontainer features discover more artifacts later. npm Git dependencies can trigger lifecycle behavior; Python sdists can pull dynamic build requirements; Cargo build-time execution can extend beyond simple fetch; workflow references can expand through reusable workflows or mutable tags [8], [27]–[29]

C. Repo-Open and Workspace Execution Surfaces

Repo-open state is in scope because opening a project can be execution-relevant. MCP definitions, hooks, memory files, startup tasks, devcontainers, and host-side initialization hooks can all shape behavior or launch follow-on actions before human review [2]–[5], [9], [30]

D. Trust Plane, Cache, and Control-Plane Compromise

The control plane becomes part of the trusted computing base. Cache poisoning, policy-store compromise, signing-key compromise, parser weakness, and stale trust roots are all relevant risks. ZitPit therefore treats resolved immutable identity, compatibility fingerprints, and any separately computed content digests as distinct inputs rather than collapsing them into one trust bit. Provenance, freshness, expiry, revocation, and operator-visible evidence remain separate obligations [11], [12], [14], [31]

E. Operator Burden and Availability

Admission control creates operational cost. Approval latency, break-glass frequency, policy drift, and availability all matter because a brittle control plane will be bypassed. The current paper therefore treats deployability as part of the security argument rather than as a UX-only concern.

V. Architecture and Policy

A. Admission Stages

ZitPit organizes the control plane into four stages:

1) **Acquire.** Resolve external requests to the strongest available immutable identity.
2) **Build.** Separate build-time or install-time execution from simple acquisition.
3) **Execute.** Grant capabilities rather than ambient trust to host execution surfaces.
4) **Publish.** Optionally inspect outgoing release artifacts and workflow outputs.

The current implementation is strongest on Git-path intake, protected-session mediation, and governed egress. Broader ecosystem adapters remain a public engineering agenda rather than hidden proof.

B. Artifact Policy Events

The admission decision is durable. Each artifact policy event records the requested selector, resolved immutable identity, provenance result, verdict, evidence pointer, context, and expiry or revocation state. That durable event is the missing join key for recall, audit, and later incident reconstruction.

Policy-event schema. selector; resolved immutable identity; provenance result; verdict; evidence pointer; context; expiry and revocation state. This is the shared contract between admission, evidence, and later recall.

TABLE I
CURRENT PUBLIC PROOF BOUNDARY

Surface	Current public evidence	Status	Supported claim
Git smart-HTTP intake	Repeated five-repository benchmark harness measuring web, disk-cache, and hot-cache latency [20]	Implemented	Approved immutable Git intake can stay faster than unmanaged public fetch
Brokered protected-session enforcement families	Docker demo + battle packs for blocked secret reads, direct egress tooling, persistence writes, publish abuse, destructive ops, and interpreter-evasion wrappers [21]	Implemented	Protected sessions can deny selected high-value command families before execution
Governed outbound DLP	Demo smoke proofs and battle packs for blocked sensitive uploads and archive scanning [21]	Implemented	Governed egress can block selected sensitive outbound data before transmission
Rust build-time execution	Battle harness coverage for `build.rs` style scenarios [21]	Partial	Build-time execution can be modeled as a separate capability boundary
GitHub Actions immutable-ref enforcement	Threat-model and scenario coverage for mutable refs and unsafe actions [21]	Partial	Workflow references should resolve to immutable identities before execution
npm / PyPI / raw installer mediation	Benchmark matrix and roadmap targets only [21]	Planned	Current paper does not claim ecosystem-complete package-manager enforcement
Repo-open enforcement depth	Threat model, policy model, and roadmap targets for `.mcp.json`, memory files, hooks, and devcontainers [21]	Planned	Repo-open state is part of the supply chain, but host-side closure is not yet fully proven

TABLE II
CAPABILITY-SCOPED VERDICTS

Verdict	Typical use	Execution conditions
`FETCH_ONLY`	First-seen dependency or repo bundle	Download allowed; protected-host execution denied
`UNPACK_ONLY`	Archive expansion and static inspection	Unpack allowed; execution denied
`BUILD_NO_NETWORK`	sdist build, `build.rs`, installer analysis	Controlled lane only; no general egress
`TEST_NO_SECRETS`	Benign validation	Isolated test lane; no sensitive material
`RUN_DEV`	Approved developer dependency	Protected-host execution under freshness policy
`RUN_CI`	Approved CI dependency or action	CI execution under stronger identity and expiry checks
`BLOCKED`	Malicious, unsupported, or stale input	Denied pending recall, expiry, or operator action

Example event. selector=`acme/tool@{pre-resolution}`; resolved immutable identity=`f3c1...`; provenance result=`verified`; verdict=`RUN_DEV`; evidence pointer=`report://quarantine`; context=`code_intake/protected_host`; expiry and revocation recorded at decision time.

C. Capability-Scoped Verdicts

Binary allow or block semantics are too coarse for agentic workflows. ZitPit uses capability-scoped verdicts so that fetching bytes, unpacking, building in quarantine, testing without secrets, and running on a protected host are separate decisions.

D. Protected-Session Enforcement Families

The repository currently demonstrates protected-session enforcement families rather than universal host guarantees. The battle packs and Docker demo cover representative pre-execution denials for interpreter-evasion wrappers, secret and key reads, SSH-agent touch, browser token access, repo-open or config abuse, publish and deploy misuse, persistence writes, destructive operations, and selected recon or lateral-movement tooling [21]. These are important proof families, but they should be described as demonstrated brokered-session controls unless host-side mandatory enforcement is fully proven.

E. Mirage Lab as Evidence, Not Oracle

Mirage Lab is useful because it improves ordering, evidence production, and operator review. It is not a safety oracle. Dynamic analysis can miss delayed triggers, sandbox-aware behavior, context-dependent payloads, or follow-on stages. The strongest statement ZitPit can make is still that unknown artifacts do not gain protected-host execution rights before policy evaluation.

F. Provenance Consumption

ZitPit is designed to consume standards-backed signals rather than replace them: TUF-style freshness and anti-rollback semantics, Sigstore identity and transparency, in-toto attestations, SLSA provenance, GitHub artifact attestations, npm trusted publishing, PyPI trusted publishing, and checksum-backed package ecosystems such as Go modules [11]–[14], [31]–[37]

VI. PRELIMINARY EVALUATION

The current evaluation is split into two proof obligations: deployability and coverage honesty.

A. Benchmark Methodology

The public timing harness now mirrors the actual upstream repository at the resolved immutable target before timing the approved path. The harness validates that the seeded managed mirror matches the claimed upstream HEAD, and it fails report generation if the managed response diverges from the expected immutable target. The mutable working outputs remain under `docs/benchmarks/latest.*`, but the paper cites a frozen benchmark snapshot artifact under `docs/benchmarks/snapshots/` so the evidence reference is not a moving "latest" pointer [20].

B. Deployability: Can the Safe Path Stay Fast?

The public benchmark harness measures the Git smart-HTTP intake path rather than a full clone or cross-ecosystem install. It times repeated `git ls-remote` / `info/refs` style mediation across five public repositories: `git`, `go`, `node`, `cpython`, and `terraform`. Each repository is measured in three modes: direct upstream request (`web`), approved disk-cache hit (`cache`), and approved hot-cache hit (`hot-cache`) [20].

This result is deliberately modest. It does not prove full clone closure, submodule closure, LFS closure, or package-manager-native mediation. It does show something operationally important: if admission control is to survive contact with real development workflows, approved artifacts cannot be slower than the public network by default.

C. Coverage Honesty: Which Attack Families Are Publicly Supported?

The coverage matrix is part of the argument, not an embarrassment to hide. A credible paper should make unsupported or partial paths visible rather than imply full closure through architectural rhetoric.

VII. Related Work

ZitPit overlaps with several adjacent control families, but it is not reducible to any one of them.

Provenance and attestations. TUF, Sigstore, in-toto, SLSA, GitHub artifact attestations, npm trusted publishing, and PyPI trusted publishing improve identity, freshness, transparency, and build-lineage statements [11]–[14], [31]–[36]. ZitPit aims to consume those signals at the moment execution rights are granted.

Workspace trust and application control. VS Code Workspace Trust, Windows application control, and platform notarization traditions already recognize that opening untrusted content or launching unsigned code can change host risk posture [9], [38], [39]. ZitPit differs by centering the consumer-side admission event for heterogeneous developer artifacts and repo-open bundles in agentic workflows.

Hermetic and reproducible builds. Bazel-style hermeticity, checksum-backed ecosystems such as Go modules, and package-manager-native integrity controls show that locality, reproducibility, and strong identity can improve both speed and trust [26], [28], [37], [40]. ZitPit extends that intuition to the broader moment where external software, repo-open state, and workflow references receive host rights.

Repository and package firewalls. Sonatype Repository Firewall, Datadog Supply-Chain Firewall, and Socket Firewall provide important ingress screening and quarantine functions [15]–[17]. ZitPit shares that concern, but pushes toward a unified admission record across intake, repo-open state, execution, and recall.

Runtime protection. Tools such as StepSecurity Harden-Runner observe or constrain execution after code begins running [18]. Those controls remain valuable, but ZitPit argues that the first durable consumer-side decision should occur earlier.

Behavioral analysis corpora. OpenSSF Package Analysis and malicious-package corpora help quantify file access, command execution, and network behavior across suspicious packages [41], [42]. ZitPit treats that style of analysis as evidence input and benchmark material, not as a replacement for admission control.

VIII. Implications

The current paper makes a narrow empirical claim and a broader architectural claim.

Empirically, the strongest public evidence today is that some important slices of intake, protected-session mediation, and governed egress can be made explicit and auditable, while approved immutable intake can remain faster than unmanaged public fetch.

Architecturally, this pattern could become a standard execution boundary for agentic environments. The next frontier is broader admission of external influence, not just packages: repo-open state, workflow references, tool manifests, and other machine-consumable context increasingly behave like supply-chain input. If admission systems become part of mainstream development infrastructure, open governance matters because these systems can centralize power if their policy memory, recall logic, and evidence formats are opaque or non-portable.

IX. Limitations and Future Work

Coverage remains incomplete. Current public proof is strongest for Git smart-HTTP intake, protected-session command families, and governed egress. Package-manager-native closure, repo-open enforcement depth, raw installer capture, and full workflow graph closure remain future work.

Mandatory mediation remains hard. Unsupported or unmanaged paths should be called unsupported, not quietly treated as secure enough.

Trusted publishers can still ship bad code. Admission control narrows the moment when rights are granted; it does not make publisher compromise harmless.

Mirage Lab is not a verifier. Dynamic analysis improves evidence and ordering but cannot prove benign behavior.

Operational cost matters. Break-glass controls, approval burden, and control-plane availability are part of the security story because systems that are too brittle are bypassed.

The near-term engineering agenda is concrete: broader package-manager-native mediation; stronger follow-on Git closure for submodules, LFS, and delayed fetches; repo-open benchmark families; stronger provenance consumption with expiry, revocation, and recall; and public benchmark suites that distinguish implemented proof from roadmap ambition.

X. Conclusion

ZitPit starts from a simple observation about agentic development: the interval between discovering external software and granting it local authority is collapsing toward zero. That changes the practical trust problem. The important question is no longer only whether a package, workflow, or repo-open bundle exists on the internet. The important question is whether

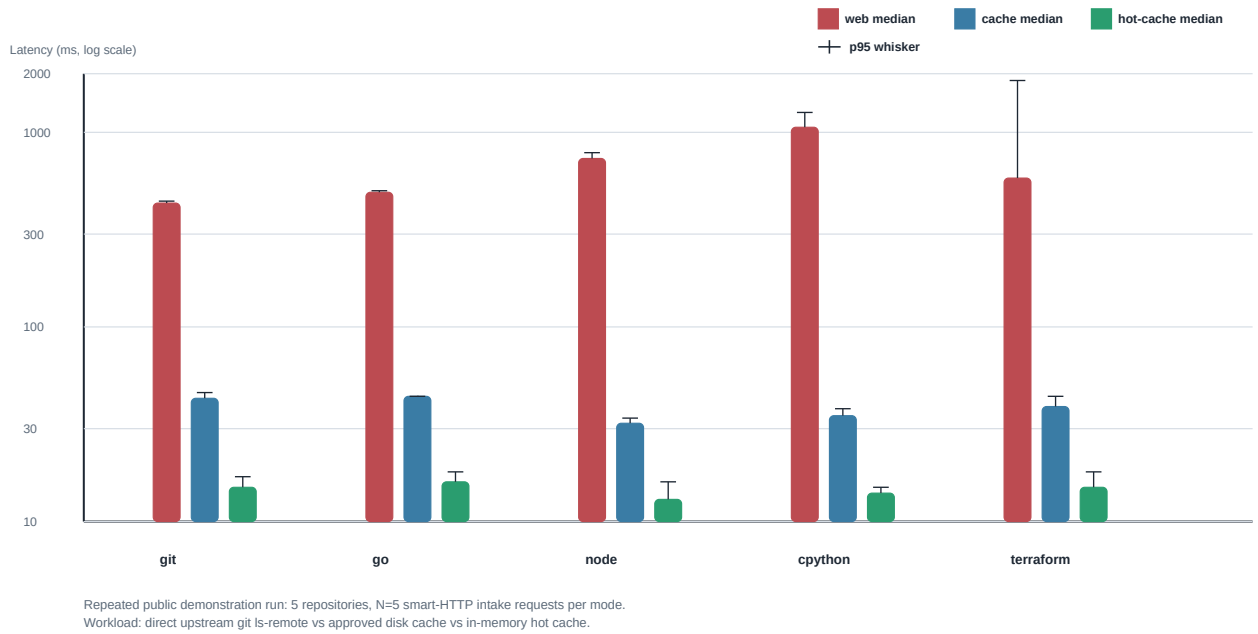

Fig. 1. Preliminary deployability result for the Git smart-HTTP intake path. Repository-level web medians ranged from 433–1062 ms, cache medians from 32–44 ms, and hot-cache medians from 13–16 ms across $N = 5$ samples per repository [20]. The significance is not universal Git closure; it is evidence that approved immutable intake can be materially faster than unmanaged public fetch.

TABLE III
ATTACK-FAMILY COVERAGE AND CURRENT PROOF STATUS

Surface	First execution boundary	Control point	Status	Evidence source	Residual risk
Git smart-HTTP intake	checkout or follow-on fetch	mediated intake and immutable ref binding	Implemented	repeated public benchmark harness [20]	submodules, LFS, and follow-on fetch are not yet fully closed
Protected-session command families	brokered shell before process start	session broker policy and battle packs	Implemented	protected-session battle packs [21]	host-side mandatory enforcement remains future work
Governed outbound DLP	proxy before upstream routing	egress broker and DLP classifiers	Implemented	smoke proofs and egress battle packs [21]	raw sockets and unmanaged egress remain outside current proof
Rust build-time execution	compile-time build script	controlled build lane	Partial	battle harnesses [21]	broader Cargo-native mediation remains future work
GitHub Actions refs and unsafe actions	workflow runner	immutable ref policy and scenario coverage	Partial	workflow scenarios [21]	reusable workflows and full graph closure remain incomplete
npm / PyPI / raw installers	install, build, or lifecycle hook	package-manager-native mediation	Planned	benchmark matrix + roadmap [21]	native package-manager closure not yet public proof
Repo-open surfaces	workspace open or tool startup	workspace policy + agent policy hooks	Planned	threat model + roadmap [21]	enforcement depth varies by IDE and runtime

it has earned execution rights in a protected environment under durable policy.

The present-tense claims in this paper are intentionally narrow. First-seen external artifacts should become policy events before they gain protected-host execution rights. Approved immutable intake can stay on a faster path than unmanaged public fetch. Current public proof is strongest for selected intake, protected-session, and governed-egress surfaces, not for universal ecosystem closure.

The larger significance is still worth stating carefully. ZitPit identifies a consumer-side admission boundary that adjacent fields have been approaching from different directions: provenance, package firewalls, workspace trust, runtime protection, and agent governance. If the agent era needs a durable place where outside software earns local authority, that boundary is a strong candidate.

REFERENCES

[1] Anthropic, "Claude code: Getting started," https://docs.anthropic.com/en/docs/claude-code/getting-started, 2026, accessed 2026-04-02.
[2] ——, "Claude code: Model context protocol (MCP)," https://docs.anthropic.com/en/docs/claude-code/mcp, 2026, accessed 2026-04-02.
[3] ——, "Claude code: Hooks reference," https://docs.anthropic.com/en/docs/claude-code/hooks, 2026, accessed 2026-04-02.
[4] ——, "Claude code: Manage claude's memory," https://docs.anthropic.com/en/docs/claude-code/memory, 2026, accessed 2026-04-02.
[5] ——, "Claude code: Devcontainer reference," https://docs.anthropic.com/en/docs/claude-code/devcontainer, 2026, accessed 2026-04-02.
[6] ——, "Claude code: Settings," https://docs.anthropic.com/en/docs/claude-code/settings, 2026, accessed 2026-04-02.

[7] GitHub Docs, "Security hardening for GitHub Actions," https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions, 2026, accessed 2026-04-02.

[8] ——, "Reuse workflows," https://docs.github.com/en/actions/sharing-automations/reusing-workflows, 2026, accessed 2026-04-03.

[9] Microsoft, "Visual studio code: Workspace trust," https://code.visualstudio.com/docs/editing/workspaces/workspace-trust, 2026, accessed 2026-04-03.

[10] ——, "Visual studio code: Copilot security," https://code.visualstudio.com/docs/copilot/security, 2026, accessed 2026-04-03.

[11] The Update Framework, "The update framework documentation," https://theupdateframework.io/, 2026, accessed 2026-04-02.

[12] Sigstore Project, "Sigstore documentation," https://docs.sigstore.dev/, 2026, accessed 2026-04-02.

[13] in-toto Project, "in-toto," https://in-toto.io/, 2026, accessed 2026-04-02.

[14] SLSA, "Supply-chain levels for software artifacts," https://slsa.dev/, 2026, accessed 2026-04-02.

[15] Sonatype, "Repository firewall," https://help.sonatype.com/en/repository-firewall.html, 2026, accessed 2026-04-02.

[16] Datadog, "Supply-chain firewall," https://github.com/DataDog/supply-chain-firewall, 2026, accessed 2026-04-02.

[17] Socket, "Socket firewall," https://socket.dev/firewall, 2026, accessed 2026-04-02.

[18] StepSecurity, "Harden-runner," https://www.stepsecurity.io/harden-runner, 2026, accessed 2026-04-02.

[19] OpenSSF, "Security-focused guide for AI code assistants," https://openssf.org/wp-content/uploads/2025/07/Security-Focused-Guide-for-AI-Code-Assistants.pdf, 2025, accessed 2026-04-02.

[20] ZitPit, "Zitpit benchmark snapshot: Five public git repositories," Repository artifact, 2026, frozen benchmark snapshot under `docs/benchmarks/snapshots/`, accessed 2026-04-04.

[21] ——, "Zitpit benchmark matrix," Repository artifact, 2026, source file: `BENCHMARKS.md`, accessed 2026-04-02.

[22] Git, "gitsubmodules documentation," https://git-scm.com/docs/gitsubmodules, 2026, accessed 2026-04-03.

[23] ——, "partial-clone documentation," https://git-scm.com/docs/partial-clone, 2026, accessed 2026-04-03.

[24] Git LFS, "Git large file storage," https://git-lfs.com/, 2026, accessed 2026-04-03.

[25] PyPA, "Vcs support," https://pip.pypa.io/en/stable/topics/vcs-support/, 2026, accessed 2026-04-03.

[26] The Rust Project, "The cargo book: Source replacement," https://doc.rust-lang.org/cargo/reference/source-replacement.html, 2026, accessed 2026-04-03.

[27] npm, Inc., "package.json," https://docs.npmjs.com/cli/v11/configuring-npm/package-json, 2026, accessed 2026-04-02.

[28] PyPA, "Secure installs," https://pip.pypa.io/en/stable/topics/secure-installs/, 2026, accessed 2026-04-03.

[29] The Rust Project, "The cargo book: Build scripts," https://doc.rust-lang.org/cargo/reference/build-scripts.html, 2026, accessed 2026-04-02.

[30] Dev Containers, "devcontainer.json reference," https://devcontainers.github.io/implementors/json_schema/, 2026, accessed 2026-04-03.

[31] J. Cappos, J. Samuel, S. Baker *et al.*, "Diplomat: Using delegations to protect community repositories," in *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, 2016, pp. 567–581. [Online]. Available: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/kaptchuk

[32] S. Torres-Arias, A. Ammula, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1393–1410. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[33] GitHub Docs, "Use artifact attestations," https://docs.github.com/en/actions/security-guides/use-artifact-attestations, 2026, accessed 2026-04-02.

[34] npm, Inc., "Trusted publishing with OIDC," https://docs.npmjs.com/trusted-publishers, 2026, accessed 2026-04-02.

[35] PyPI Docs, "Publishing to pypi with a trusted publisher," https://docs.pypi.org/trusted-publishers/, 2026, accessed 2026-04-02.

[36] ——, "Pypi publish attestation (v1)," https://docs.pypi.org/attestations/publish/v1/, 2026, accessed 2026-04-02.

[37] The Go Authors, "The go checksum database," https://go.dev/ref/mod#checksum-database, 2026, accessed 2026-04-02.

[38] Microsoft Learn, "Application control for windows," https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/appcontrol, 2026, accessed 2026-04-03.

[39] Apple Developer, "Safely open apps on your mac," https://support.apple.com/guide/mac-help/open-a-mac-app-from-an-unidentified-developer-mh40616/mac, 2026, accessed 2026-04-03.

[40] Bazel, "Hermeticity," https://bazel.build/basics/hermeticity, 2026, accessed 2026-04-03.

[41] OpenSSF, "Package analysis," https://package-analysis.com/, 2026, accessed 2026-04-03.

[42] ——, "Malicious packages repository," https://github.com/ossf/malicious-packages, 2026, accessed 2026-04-03.